Project 2 — ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses

Title: ResearchHand Tool Agent Get Verified Answers Not Guesses

Type: Multi-agent (2 agents) · **Stack:** Python, LangChain (LCEL + tools), an agent loop you build yourself · **Level:** Beginner–Intermediate

Tagline: A tool-using Worker agent paired with a Verifier agent that checks the Worker's answer before it ever reaches the user.

> This is the **reading guide** — the story, the folder structure and why each part exists, setup, the 4 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/` .

Charter (what this project is)

Two agents: a **Worker** (3 real tools, a hard step limit, recovers cleanly from a tool failure) and a **Verifier** (a second, separate check that compares the Worker's answer with what the tools really returned, before it goes out). This is where "agent" stops being a buzzword — you build the loop yourself — and where you get your first creator-and-checker pair, the same shape Project 4's Writer/Reviewer builds on.

Jump to: Read first · Project structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4 · Checklist · Publish

The Story — what you're actually building

Think about asking a smart friend a factual question. If they guess from memory, they can sound confident and still be wrong. A friend you'd actually trust looks things up, does the maths instead of estimating it, and — for anything important — has a second person check the answer before it reaches you.

That's what you're building. The **Worker** is the friend who looks things up: given a task, it calls real tools (live weather, a converter, a policy search) instead of guessing, step by step, with a hard limit so it can never spin forever. The **Verifier** is the second person: it reads the question, the answer and

what the tools actually returned, and asks "does this really hold up?" — and if not, it says so instead of letting a shaky answer through.

Each Step builds one piece: Step 1 proves a plain LCEL chain works — and shows it can only guess. Step 2 adds one tool and a hand-built loop — the smallest real agent. Step 3 grows it into the real Worker with 3 tools, a limit and failure recovery. Step 4 adds the Verifier and completes the two-agent handoff.

What you're building, in one line: a Worker agent that calls real tools in a loop to answer a question, plus a separate Verifier agent that checks the answer against the tools' evidence before it goes out.

Why this needs to exist: a model answering from memory states a wrong answer as confidently as a right one. When a wrong answer costs something, you need an agent that looks things up — and a second one that checks the result.

When you'd reach for this at a real job: when an assistant must answer with live or exact information — a price, a stock level, a policy, a calculation — instead of a memorized guess.

How it works, mechanically: the Worker runs a think → act → observe loop with a step limit, calling tools as needed. When it stops, the Verifier gets the question, the answer and the tools' results, and returns a checked `approved` / `reason` verdict. A rejected answer gets one retry.

Why not skip the loop and give the model all tool results upfront? You often don't know in advance which tool, or which follow-up lookup, a question needs — the loop lets the model decide one step at a time. **Why not let the Worker check its own answer?** A model checking its own work shares the blind spots that made the mistake; a separate Verifier with its own prompt catches more.

Where This Fits in the 5-Project Arc

```
Project 1               → 2 agents, Python router, raw SDK
                          (Concierge + Triage)
Project 2 (you are here) → 2 agents, hand-built agent loop, tools
                          (Worker + Verifier)
Project 3               → 3 agents, LangGraph, RAG + human approval
Project 4               → 4-5 agents, one Supervisor directs the rest
Project 5               → Project 4, made production-ready
```

Project 1's agents never used tools. Project 2's new idea is an agent that **acts** — and a second agent that **checks** it.

What to Read First (docs this project uses)

Everything this project needs was taught in Docs 01-07. Use this table when a step says "Read first", or whenever an idea feels unfamiliar:

You need to understand…	Read this	Used in
`load_config()`, `get_logger()`	Doc01 Build Task	Step 1
`request_with_retry()`, HTTP errors	Doc02 Build Task	Step 3
Which API errors to retry	Doc04 — Error types	Steps 1, 3
`prompt | model | parser` (LCEL)	Doc05 — Core Concepts + LCEL basics exercise	Steps 1, 4
`with_structured_output(Model)`	Doc05 — structured parser exercise	Step 4
`@tool`, `bind_tools`, `tool_calls`	Doc06 — first tool exercise	Step 2
A tool's description is a prompt	Doc06 — descriptions	Steps 2, 3
Arguments checked by Pydantic	Doc06 — Pydantic contract	Steps 2, 3
The 5 steps of one tool round-trip	Doc06 — three ways	Step 2
A real-API tool (Open-Meteo)	Doc06 — real API exercise	Step 3
A tool failure the model can see	Doc06 — failure path	Step 3
The think → act → observe loop	Doc07 — ReAct loop	Steps 2, 3
Step limits, `MaxIterationsExceeded`	Doc07 — step limit	Step 3
The whole Worker, as a practice run	Doc07 Build Task	Step 3
Generator → critic (look ahead)	Doc11 — Patterns	Step 4

Reused code, unchanged except for its folder and import lines: `config.py`, `exceptions.py` and `utils/logger.py` (Doc01), `utils/http_client.py` (Doc02, logger moved inside the function), `find_tool()` and the weather tool (Doc06), `AgentResult`, `run_tool()`, `MaxIterationsExceeded` and `FlakyToolState` (Doc07), and `utils/prompts.py` (Project 1).

Project Structure (the professional layout)

Same rule as Project 1: keep **only the folders this project has a real job for**. Project 2 adds two folders Project 1 didn't need — `tools/` and `chains/`.

Names: the repo folder vs. the Python package

- **Repo folder:** `ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/` — already created for you, and the exact name of the GitHub repo.
- **Python package:** `src/`, right next to `main.py`. It holds all the app's code, and every file imports from it the same way: `from src.agents.worker import run_worker`.
- **Files:** `snake_case.py`, one job each. Test files start with `test_`. Each tool file is named for what it does (`weather.py`, not `tool2.py`).
- **Prompt files:** named after who uses them (`worker.txt` → `agents/worker.py`).

The final layout (after Step 4)

```
ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/
├── data/
│   └── sample/
│       └── tasks.json          sample tasks for the script
├── scripts/
│   └── compare_with_tools.py   memory answer vs. verified answer
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── verifier.py     verify() -> VerifierVerdict
│   │   └── worker.py       run_worker() -> AgentResult     (Doc07)
│   ├── chains/
│   │   ├── __init__.py
│   │   └── direct_answer.py build_direct_chain() — no tools
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py          get_model() -> ChatOpenAI
│   ├── prompts/
│   │   ├── direct.txt      the no-tools chain's prompt
│   │   ├── verifier.txt    the Verifier's prompt
│   │   └── worker.txt      the Worker's system prompt
│   ├── services/
│   │   ├── __init__.py
│   │   └── research_service.py answer_task(): Worker -> Verifier
│   ├── tools/
│   │   ├── __init__.py
│   │   ├── converter.py    celsius_to_fahrenheit (pure logic)
│   │   ├── docs_search.py  search_docs (fails on 1st call)
│   │   └── weather.py      get_weather (real API)          (Doc06)
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── http_client.py  request_with_retry()           (Doc02)
│   │   ├── logger.py       get_logger()                   (Doc01)
```

```
│   │   └── prompts.py      load_prompt(name)          (Project 1)
│   ├── __init__.py
│   ├── config.py           Config, load_config()          (Doc01)
│   ├── exceptions.py       all named errors  (Doc01/02/07)
│   └── schemas.py          VerifierVerdict
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py real API calls
│   └── unit/
│       ├── test_agents.py      no API key, no network
│       ├── test_config.py      no API key, no network
│       └── test_tools.py       no API key, no network
├── .env                    your real key — never committed
├── .env.example            the setting names, no real values
├── .gitignore              keeps .env, .venv/, __pycache__/ out of git
├── LICENSE                 lets others reuse your code (MIT)
├── main.py                 the terminal loop — the file you run
├── README.md               the repo's front page (copied in at the end)
└── requirements.txt        the 5 libraries, for pip install -r
```

Why each folder exists

- **`data/sample/`** — the sample tasks the script reads, stored once as JSON.
- **`scripts/`** — tools you run by hand, not part of the app. **Why:** `compare_with_tools.py` prints answers for a *person* to judge.
- **`src/`** — the package: every other `.py` file the app is made of. **Why:** one name to import from, everywhere — `main.py`, tests and scripts all say `from src... import ...`.
- **`agents/`** — one file per agent (Worker, Verifier). **Why:** two agents with two different jobs and two different prompts — separate files make "these are 2 agents" something you can see.
- **`chains/`** — plain LCEL chains with no tools. **Why:** a chain and an agent are different things; keeping them apart makes that visible. Here it holds Step 1's "answer from memory" chain, which the compare script uses to show what tools add.
- **`models/`** — the one place that builds the `ChatOpenAI` model. **Why:** every chain and agent gets the same model and settings; changing the model is one line. ("Models" means *language models* — Pydantic models live in `schemas.py`.)
- **`prompts/`** — every prompt as a `.txt` file. **Why:** you can reword how an agent behaves without touching Python, and all prompts are readable in one place.
- **`services/`** — the flow between agents: Worker → Verifier → maybe one retry. **Why:** `main.py` only reads input and prints; the flow can be called from a test, a script or (in Project 5) a web API.
- **`tools/`** — one file per tool. **Why:** tools grow and change on their own (a new API, a new city); separate files keep each one small and testable. The Worker imports them; nothing else needs to know how they work.
- **`utils/`** — helpers used by several files: logging, the HTTP client, the prompt loader.

- **`tests/integration/`** — calls the real model and weather API. **Why:** costs money and needs a key, so it's kept apart and run on purpose.
- **`tests/unit/`** — runs with no API key and no network, in about a second. **Why:** run it after every change.
- **`main.py`** (a file, at the root) — the one file you run: `python main.py`. **Why:** it sits right next to the `src/` folder, so Python finds the package there.

Why each root file exists

- `.env.example` — which settings to create, with no real values.
- `.gitignore` — stops `.env`, the venv and cache files from being pushed.
- `LICENSE` — the MIT license text, created in Setup. Without one, nobody may legally reuse your code.
- `README.md` — what someone sees first on GitHub.
- `requirements.txt` — the library list for `pip install -r requirements.txt`.

Why each file exists (what and why)

Every file you'll write, in one place — the same What/Why list the documents give after each tree. The steps below build these files one at a time; come back here whenever you forget why a file exists.

- `data/sample/tasks.json`
  - **What:** A JSON object with a `"tasks"` list of sample questions.
  - **Why:** The script uses the same inputs every run, so you can compare results over time.
- `scripts/compare_with_tools.py`
  - **What:** One `main()` that reads `tasks.json` and, for each task, prints the direct chain's answer, then `answer_task()`'s answer and verdict.
  - **Why:** It shows the project's whole point in one run — a guess next to a checked answer — for you to judge.
- `agents/verifier.py`
  - **What:** Three functions: `build_verifier_chain(model)` returns `prompt | model.with_structured_output(VerifierVerdict)`; `collect_evidence(steps)` joins only the successful tool results; `verify(chain, question, result)` invokes the chain and logs the verdict.
  - **Why:** A second agent with its own prompt checks the answer against the tools' evidence, and catches a confident wrong answer before the user sees it.
- `agents/worker.py`
  - **What:** Written once in Step 2 and only extended after: `ALL_TOOLS` (one tool in Step 2, three from Step 3), an `AgentResult` dataclass (`final_answer`, `steps`), `find_tool(tools, name)`, `run_tool(call, tools)` (returns `(result, None)` or `(None, error)`), and

`run_worker(model, task, max_iterations)`: bind the tools, then loop — invoke, append the reply, stop if there are no tool calls, otherwise run each call, append a `tool` message, record a step — and raise `MaxIterationsExceeded` after the last round.
    - **Why:** This is the agent that uses the tools. The step limit means it never runs forever, and the step log shows exactly how it reached its answer.
- `chains/direct_answer.py`
    - **What:** One function, `build_direct_chain(model)`, which returns `prompt | model | StrOutputParser()` with the direct prompt.
    - **Why:** It answers from memory with no tools. The compare script uses it to show the difference between a guess and a checked answer.
- `models/llm.py`
    - **What:** A `MODEL_NAME` constant and `get_model(config)`, which returns `ChatOpenAI(model=..., temperature=0, api_key=config.openai_api_key)`.
    - **Why:** Every chain and agent uses the same model and settings.
- `prompts/*.txt`
    - **What:** Three text files: the direct prompt (with `{question}`), the Worker's system prompt (when to use which tool, retry once, answer only from tool results), and the Verifier's template (with `{question}`, `{evidence}`, `{answer}`).
    - **Why:** Each agent's instructions are in one readable place, apart from the code.
- `services/research_service.py`
    - **What:** One function, `answer_task(model, verifier_chain, task, max_iterations)`. It runs `run_worker()`, then `verify()`. If the answer is rejected, it adds the reason to the task, runs the Worker **once** more, verifies again against the original task, and returns `(result, verdict)`.
    - **Why:** The whole handoff between the two agents is one testable function, and a rejected answer gets exactly one chance to be fixed.
- `tools/converter.py`
    - **What:** One `@tool` function, `celsius_to_fahrenheit(celsius: float) -> float`, which returns `round(celsius * 9 / 5 + 32, 1)`. Its docstring says when to use it.
    - **Why:** The model gets an exact result instead of guessing the maths. `@tool` also checks the argument type before the function runs.
- `tools/docs_search.py`
    - **What:** A `COMPANY_DOCS` dict, a `FlakyToolState` class with a `call_count`, and a `@tool` function `search_docs(query)` that raises `ConnectionError` on the first call and afterwards returns the policy text whose key appears in the query.
    - **Why:** It fails on purpose, so every run proves the Worker recovers from a tool failure instead of crashing.
- `tools/weather.py`
    - **What:** A `CITY_COORDINATES` dict, `build_weather_params(city)` (returns the latitude/longitude params or `None`), and a `@tool` function `get_weather(city)` that calls

`request_with_retry("GET", WEATHER_URL, params=...)` and returns the temperature as text.
    - **Why:** The model can't know today's weather from its training. This gives it live data, with timeouts and retries from Doc02.
- `utils/http_client.py`
    - **What:** Doc02's `request_with_retry(method, url, max_attempts=5, **kwargs)`: a `for` loop that calls `requests.request(...)` with a timeout, returns `response.json()` on success, sleeps with `compute_backoff_delay()` and retries on timeouts, 429 and 5xx, and raises `PermanentHTTPError` on other 4xx.
    - **Why:** The weather tool gets timeouts and retries without writing them again.
- `utils/logger.py`
    - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
    - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
    - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
    - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `__init__.py` (every code folder)
    - **What:** An empty file. You create one in every folder that holds Python code — nothing goes inside it.
    - **Why:** Python only treats a folder as a package when this file is there. Without it, an import like `from <package>.<folder> import ...` fails with `ModuleNotFoundError`.
- `config.py`
    - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
    - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
    - **What:** Four error classes: `MissingConfigError` (Doc01), `TransientHTTPError` and `PermanentHTTPError` (Doc02), and `MaxIterationsExceeded` (Doc07), whose `__init__` stores the partial `steps` list.
    - **Why:** Each problem can be caught by name. `MaxIterationsExceeded` carries the step log, so hitting the limit still shows where the Worker got stuck.

- `schemas.py`
  - **What:** One Pydantic class, `VerifierVerdict`, with `approved: bool` and `reason: str`.
  - **Why:** The Verifier's answer is a real True/False the code can use. Without it you'd search the text for "approved" — and "NOT approved" would pass.
- `tests/integration/test_live_agents.py`
  - **What:** Test functions that call the real model (and the weather API), added step by step: the no-tools chain, the right tool for a clear task, no tool for general knowledge, the step limit, a weather task, recovery from the failing search, and the Verifier rejecting a wrong answer and approving a right one. Each prints `OK` or `WRONG`.
  - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `tests/unit/test_agents.py`
  - **What:** One test function per case, each printing `OK` or `WRONG`: hand-made tool-call dicts for `find_tool()` and `run_tool()` (known tool, unknown tool, good call, unknown-tool call, bad arguments), and hand-made steps for `collect_evidence()` (a failed step left out, no steps).
  - **Why:** It proves failures turn into readable error messages and only good results reach the Verifier — without any model call.
- `tests/unit/test_config.py`
  - **What:** Three test functions, no API call, each printing `OK` or `WRONG`: `require_env()` raises `MissingConfigError` for a missing and for an empty setting (a made-up setting name, so your `.env` is never touched), and `load_config()` returns a `Config` when a key is set.
  - **Why:** It proves a missing key stops the app with one clear error, with no API call — the first thing that must work.
- `tests/unit/test_tools.py`
  - **What:** One test function per case, each printing `OK` or `WRONG`, calling the tools directly with `.invoke({...})`: 18°C is 64.4°F, a bad argument raises `ValidationError`, the weather params for a known and an unknown city, and the search failing once and then working.
  - **Why:** A broken tool shows up here, with no network and no model — not as a strange agent answer later.
- `main.py`
  - **What:** `main()` loads the config, builds the model and the Verifier chain, then loops: read a task, call `answer_task()`, print the step log with `print_steps()`, and print the answer marked verified or NOT verified. It catches `MaxIterationsExceeded` (prints the partial log) and the OpenAI key and rate-limit errors.
  - **Why:** One starting point that reads input and prints results. The agents and the flow live in other files.

How imports work in this layout

- Every folder with Python files gets an empty `__init__.py`. It marks the folder as a package, so `src.agents.worker` is importable. (`prompts/` holds only `.txt` files, so it doesn't need one.)
- Python looks for imports in the folder you **run from**. `main.py` and `src/` sit side by side in the repo folder, so `python main.py` finds `from src.tools.weather import get_weather` right there.
- Tests and scripts live in sub-folders, so run them as modules from the repo folder: `python -m tests.unit.test_tools`. The `-m` tells Python to start from the folder you're in — the repo folder — so `src` is found there too.
- Always run commands from the **repo folder**. The prompt loader, the tests and the script also use paths that start there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
`api/` , `routes.py`	A terminal app, no web server	Project 5 (Doc12)
`rag/` , `vectorstore.py`	`search_docs` is a small local lookup, not real retrieval	Project 3 (Doc08)
`graph/` , LangGraph	The loop is hand-built on purpose	Project 3 (Doc09)
`configs/*.yaml`	2 settings fit in `.env`	When settings grow
`Dockerfile` , `Makefile` , CI	Nothing to deploy yet	Doc12-13
`tests/conftest.py` , pytest	Plain test scripts, same style as Project 1	Doc13
A frontend	A terminal app is enough to show the idea	Streamlit, when a project needs one

How the structure grows, step by step

Step	New files	Changed files
Setup	root files, `src/__init__.py`	—
1	`config.py` , `exceptions.py` , `utils/logger.py` , `utils/prompts.py` , `models/llm.py` , `chains/direct_answer.py` , `prompts/direct.txt` , `main.py` , `tests/unit/test_config.py` , `tests/integration/test_live_agents.py`	—

Step	New files	Changed files
2	`tools/converter.py`, `agents/worker.py`, `prompts/worker.txt`, `tests/unit/test_tools.py`, `tests/unit/test_agents.py`	`exceptions.py`, `main.py`, `test_live_agents.py`
3	`utils/http_client.py`, `tools/weather.py`, `tools/docs_search.py`	`exceptions.py`, `agents/worker.py` (tool list only), `prompts/worker.txt`, `test_tools.py`, `test_live_agents.py`
4	`schemas.py`, `agents/verifier.py`, `prompts/verifier.txt`, `services/research_service.py`, `scripts/compare_with_tools.py`, `data/sample/tasks.json`	`main.py`, both test files in `tests/`

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files. The steps are the same as Project 1's Setup, with a different package name and library list.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`) that's:

```
cd project_2_researchhand_tool_agent
cd ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Create `.gitignore` **before** your first commit:

```
# secrets — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
# Python's compiled cache files
__pycache__/
*.pyc
```

3. Create `.env` (your real key) and `.env.example` (same names, no values):

```
# .env — never commit this file
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

```
# .env.example — commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

4. Create `requirements.txt`:

```
langchain-core>=1.0
langchain-openai>=1.0
pydantic>=2.0
python-dotenv>=1.0
requests>=2.31
```

`langchain-openai` brings the `openai` library with it. `requests` is for Doc02's HTTP client (the weather tool).

5. Create an empty `src/__init__.py`, then install the libraries:

```
pip install -r requirements.txt
```

Check it worked: from the repo folder, `python -c "import src; print('ok')"` prints `ok`. It works because the `src/` folder sits right there — that's why every command runs from the repo folder.

6. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
```

```
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Why This Project Is Good for Your Portfolio

- **What problem it solves:** a model that can *act* — look up live data, calculate, search — and a second model that checks the result. That's the difference between a chatbot and a tool you'd trust.
- **Why it matters:** the tool-using loop is the building block of every later agent system (supervisors, RAG agents, multi-agent teams). A hand-built loop shows you understand it, not just call it.
- **When you'd build something like this at a job:** any assistant that checks live data, calls internal services or does multi-step lookups before answering.
- **How it's built:** a think → act → observe loop with a hard limit, 3 Pydantic-checked tools, a step log for every run, and an independent Verifier with a structured verdict.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The agent keeps looping on an unclear task	The hard `max_iterations` limit raises `MaxIterationsExceeded` with the partial log — never trust the model to stop on its own
The model ignores a tool's error and invents a result	Return errors as clear `Error: ...` observations, and say in `worker.txt` to answer only from what the tools returned
Two tools with overlapping jobs get mixed up	Sharpen the tool descriptions — say when *not* to use each one. It's a prompt fix, not a code fix
The Verifier approves everything	Give it the tools' evidence, a strict separate prompt, and test it with a deliberately wrong answer (Step 4, case 5)
`400` error about `tool_call_id`	Append the model's own reply to `messages` **before** the tool results — every time

Problem	Fix
`ModuleNotFoundError: No module named 'src'`	You're not in the repo folder, or you ran a test as `python tests/...` instead of `python -m tests....`

Built During These Documents

05_langchain_fundamentals → 06_tools_function_calling → 07_ai_agents (on top of Docs 01-04)

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks before writing the agent loop.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 answers from memory, Step 2 is a one-tool agent, Step 3 is the full Worker, Step 4 adds the Verifier. Each step only adds or changes a few files. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command runs from the **repo root** (`ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/`) with the venv active.

How testing works in this project

Story: ResearchHand's whole promise is "verified answers, not guesses" — so its own code gets the same treatment. Every step adds the tests for the feature it adds: the chain, the tool loop, each tool, the step limit, the Verifier. When something breaks, the newest `WRONG` points straight at the newest code. **If not:** you'd only ever know the agent "seemed to work" on the questions you happened to type, and a tool that quietly broke would show up as a confident wrong answer.

- **Two kinds of test:**
  - `tests/unit/` — code that needs no model and no network: settings, each tool called directly with `.invoke({...})`, the Worker's and Verifier's helper functions. No API key, no cost, under a second. **When:** after every change to the file it tests — run them all the time.
  - `tests/integration/test_live_agents.py` — the parts that need the real model (and the weather API): the chain, the Worker's tool choices, the step limit, recovery, the Verifier. Real API calls (a fraction of a cent). **When:** at the end of every step, and after changing a prompt, a tool's docstring, or `MAX_ITERATIONS` in `main.py`.
- **One pattern for every case** — so the output never needs judging:

```
def test_something() -> None:
    result = "OK"
    if <it came out wrong>:
        result = "WRONG"
    print(f"case 1 (what it checks) -> {result}")
```

- **An error you expect** (like `ValidationError` or `MaxIterationsExceeded`): start with `result = "WRONG"`, and set `result = "OK"` inside `except TheError:`.
- **One function per case**, all called at the bottom under `if __name__ == "__main__":`. Each file numbers its own cases, and case numbers never change once written.
- **Model answers vary**, so check one fact — a tool name in the step log, "Paris" in the answer, `approved` is `False` — never the exact wording.
- **Run from the repo root with `-m`:** `python -m tests.unit.test_tools` (dots, no `.py`). Running the file by its path fails with `No module named 'src'`, because Python then starts looking for imports in `tests/unit/`.
- **Got a `WRONG`?** Read that one test function, then the code it calls. Fix the code — change the test only if the test itself checks the wrong thing.

Step 1 — A Single LCEL Chain That Answers One Question, No Tools

Project: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — Step 1 of 4

- **Where you are now:** Setup is done — repo folder, venv, `.env`, `requirements.txt`, and an empty `src/` package. Nothing runs yet.
- **What's new in this step:** Doc01's config/logging files, `models/llm.py`, the prompt loader from Project 1, and Doc05's `prompt | model | parser` chain in `chains/direct_answer.py`, with a question loop in `main.py`, and the first tests.
- **Why:** prove the LangChain pieces work on their own, before any tool or loop. It also shows the problem: a live question gets a guess.
- **How:** `load_config()` → `get_model()` → `build_direct_chain()` → `chain.invoke({"question": ...})` → print.
- **Working after this step:** `python main.py` answers questions from the model's memory, and both test files pass.

Read first: Doc05 — LCEL basics exercise, Doc01 Build Task.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Bring in Doc01's `exceptions.py`, `config.py` and `logging_setup.py` (as `utils/logger.py`), and Project 1's `utils/prompts.py`. Change only the import lines and the prompts folder path (`src/prompts`).

2. Write `models/llm.py`: `get_model(config)` returns `ChatOpenAI(model=..., temperature=0, api_key=config.openai_api_key)`.
3. Write `prompts/direct.txt` (with a `{question}` placeholder) and `chains/direct_answer.py`: `build_direct_chain(model)` returns `prompt | model | StrOutputParser()`.
4. Write `main.py`: config, model, chain, then a loop — `/quit` to exit, and a wrong key gives one clear line (`openai.AuthenticationError`).
5. Ask a general question, then a live one ("What's the weather in Lahore right now?"). See the guess.
6. Write `tests/unit/test_config.py` (Doc01's three config checks, each printing `OK` or `WRONG`) and `tests/integration/test_live_agents.py` with case 1 (the chain answers a question). Run both with `python -m`.

Your files after Step 1:

```
ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/
├── src/
│   ├── chains/
│   │   ├── __init__.py      (new)
│   │   └── direct_answer.py (new) build_direct_chain()
│   ├── models/
│   │   ├── __init__.py      (new)
│   │   └── llm.py           (new) get_model()
│   ├── prompts/
│   │   └── direct.txt       (new)
│   ├── utils/
│   │   ├── __init__.py      (new)
│   │   ├── logger.py        (new) get_logger()
│   │   └── prompts.py       (new) load_prompt()
│   ├── __init__.py
│   ├── config.py            (new) Config, load_config()
│   └── exceptions.py        (new) MissingConfigError
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py (new) case 1, real API
│   └── unit/
│       └── test_config.py   (new) no API key needed
├── .env                (Setup)
├── .env.example        (Setup)
├── .gitignore          (Setup)
├── LICENSE             (Setup)
├── main.py             (new) question loop -> direct chain
└── requirements.txt    (Setup)
```

Why these files (what and why):

- `chains/direct_answer.py`
  - **What:** One function, `build_direct_chain(model)`, which returns `prompt | model | StrOutputParser()` with the direct prompt.
  - **Why:** It answers from memory with no tools. The compare script uses it to show the difference between a guess and a checked answer.
- `models/llm.py`
  - **What:** A `MODEL_NAME` constant and `get_model(config)`, which returns `ChatOpenAI(model=..., temperature=0, api_key=config.openai_api_key)`.
  - **Why:** Every chain and agent uses the same model and settings.
- `prompts/direct.txt`
  - **What:** This text file holds the direct prompt. The code loads it with `load_prompt("direct")`.
  - **Why:** Each agent's instructions are in one readable place, apart from the code.
- `utils/logger.py`
  - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
  - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
  - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
  - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `config.py`
  - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
  - **What:** One error class, `MissingConfigError` (Doc01), raised by `load_config()` when a setting is missing.
  - **Why:** Each problem can be caught by name. `MaxIterationsExceeded` carries the step log, so hitting the limit still shows where the Worker got stuck.
- `test_live_agents.py`
  - **What:** One test function that calls the real model: `test_direct_answer()` builds the direct chain, asks for the capital of France, and prints `OK` only if the answer contains "Paris" (case

1).
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `test_config.py`
    - **What:** Three test functions, no API call, each printing `OK` or `WRONG`: `require_env()` raises `MissingConfigError` for a missing and for an empty setting (a made-up setting name, so your `.env` is never touched), and `load_config()` returns a `Config` when a key is set.
    - **Why:** It proves a missing key stops the app with one clear error, with no API call — the first thing that must work.
- `main.py`
    - **What:** One function, `main()`. It loads the config, builds the model and the direct chain, then loops: read a question, stop on `/quit`, and print the chain's answer from memory. A wrong key gives one clear line.
    - **Why:** One starting point that reads input and prints results. The agents and the flow live in other files.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 1 (missing setting)** — `tests/unit/test_config.py`
    - **Check:** with a made-up setting name removed from `os.environ`, `require_env()` raises `MissingConfigError`.
    - **Why:** a missing key must stop the app at startup with one clear line.
    - **When:** after any change to `config.py` or `exceptions.py`.
    - **If not:** a missing key would surface later as a confusing `AuthenticationError` from OpenAI.
- **case 2 (empty setting)** — `tests/unit/test_config.py`
    - **Check:** the same name set to `""` also raises `MissingConfigError`.
    - **Why:** `OPENAI_API_KEY=` with nothing after it is "set" but never usable.
    - **When:** same as case 1.
    - **If not:** the app would start happily and fail on the first real call.
- **case 3 (config loads)** — `tests/unit/test_config.py`
    - **Check:** with `OPENAI_API_KEY` set in `os.environ`, `load_config()` returns a `Config` holding that key.
    - **Why:** cases 1–2 only prove the *error* path; this proves the values actually come back.
    - **When:** same as case 1.
    - **If not:** a missing `return value` in `require_env()` passes cases 1–2, and every setting silently becomes `None`.
- **case 1 (direct answer)** — `tests/integration/test_live_agents.py`
    - **Check:** the no-tools chain, asked "What is the capital of France?", answers with "Paris".

- **Why:** proves the key, the model, the prompt file and the LCEL chain work together, end to end.
- **When:** at the end of the step, and after changing `prompts/direct.txt` or `models/llm.py`.
- **If not:** a broken prompt variable or model name would be found by the first user, not by you.

Step 2 — One Tool, One Loop: the Smallest Possible Working Agent

Project: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — Step 2 of 4

- **Where you are now:** after Step 1 the app answers only from memory — it can't look anything up or calculate reliably.
- **What's new in this step:** one tool (`tools/converter.py`), the Worker's **finished** loop (`agents/worker.py` : `AgentResult` , `find_tool()` , `run_tool()` , a step log and a hard step limit), `MaxIterationsExceeded` , its prompt, and the first tests.
- **What stays the same:** config, logging, `get_model()` , the prompt loader, and the direct chain (no longer used by `main.py` , but kept).
- **Why:** an agent is a model in a loop that can ask for tools. You build the real loop now, with one pure-logic tool, so every line is clear — and Step 3 only has to add tools to it.
- **How:** `bind_tools(ALL_TOOLS)` → invoke → append the model's reply → run each `tool_call` through `run_tool()` (any failure becomes an `Error: ...` observation) → append a `{"role": "tool", ...}` result and a step record → invoke again, until there are no `tool_calls` or `max_iterations` is reached.
- **Working after this step:** ask "What is 25°C in Fahrenheit?" — the model calls the converter, you see the step log, the Worker can never loop forever, and all tests pass.

Read first: Doc06 — first tool exercise, Doc06 — three ways, Doc07 — ReAct loop.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `tools/converter.py` : a `@tool` function `celsius_to_fahrenheit(celsius: float)` , with a docstring that says when to use it (Doc06: the description is a prompt).
2. Write `prompts/worker.txt` : use the tool for every conversion, answer everything else directly.
3. Add Doc07's `MaxIterationsExceeded` to `exceptions.py` , then write `agents/worker.py` : `ALL_TOOLS = [celsius_to_fahrenheit]` , the `AgentResult` dataclass, `find_tool()` , `run_tool()` and `run_worker(model, task, max_iterations)` — Doc07's loop with a step record and a log line per call, raising `MaxIterationsExceeded` after the last round.
4. Change `main.py` : call `run_worker()` instead of the chain, print the step log with `print_steps()` , and catch `MaxIterationsExceeded` and `openai.RateLimitError` .
5. Write `tests/unit/test_tools.py` (a normal conversion, and a bad argument that `@tool` rejects), `tests/unit/test_agents.py` (`find_tool()` / `run_tool()` with hand-made calls)

and add cases 2-4 to `tests/integration/test_live_agents.py` (tool used, no tool, step limit).

6. Test one prompt that needs the tool and one that doesn't. Check the log: the tool runs only when it should.

Your files after Step 2:

```
ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/
├── src/
│   ├── agents/
│   │   ├── __init__.py      (new)
│   │   └── worker.py        (new) the finished loop, 1 tool
│   ├── prompts/
│   │   └── worker.txt       (new)
│   ├── tools/
│   │   ├── __init__.py      (new)
│   │   └── converter.py     (new) celsius_to_fahrenheit
│   ├── (Step 1's other files, unchanged)
│   └── exceptions.py        (changed) + MaxIterationsExceeded
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py (changed) + cases 2-4
│   └── unit/
│       ├── test_agents.py   (new) find_tool, run_tool
│       └── test_tools.py    (new) no API key needed
└── main.py             (changed) run_worker(), step log, errors
```

Why these files (what and why):

- `agents/worker.py`
  - **What:** Written once in Step 2 and only extended after: `ALL_TOOLS` (one tool in Step 2, three from Step 3), an `AgentResult` dataclass (`final_answer` , `steps`), `find_tool(tools, name)` , `run_tool(call, tools)` (returns `(result, None)` or `(None, error)`), and `run_worker(model, task, max_iterations)` : bind the tools, then loop — invoke, append the reply, stop if there are no tool calls, otherwise run each call, append a `tool` message, record a step — and raise `MaxIterationsExceeded` after the last round.
  - **Why:** This is the agent that uses the tools. The step limit means it never runs forever, and the step log shows exactly how it reached its answer.
- `prompts/worker.txt`
  - **What:** This text file holds the worker prompt. The code loads it with `load_prompt("worker")` .
  - **Why:** Each agent's instructions are in one readable place, apart from the code.
- `tools/converter.py`

    - **What:** One `@tool` function, `celsius_to_fahrenheit(celsius: float) -> float`, which returns `round(celsius * 9 / 5 + 32, 1)`. Its docstring says when to use it.
    - **Why:** The model gets an exact result instead of guessing the maths. `@tool` also checks the argument type before the function runs.
- `exceptions.py`
    - **What:** Two error classes: `MissingConfigError` (Doc01) and `MaxIterationsExceeded` (Doc07), which carries the partial step log.
    - **Why:** Each problem can be caught by name. `MaxIterationsExceeded` carries the step log, so hitting the limit still shows where the Worker got stuck.
- `test_live_agents.py`
    - **What:** Step 1's case plus `tools_used()` and three Worker cases with the real model: a conversion uses the converter, a general question uses no tool, and one round can't finish a tool task, so `MaxIterationsExceeded` is raised (cases 2-4). Each prints `OK` or `WRONG`.
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `test_agents.py`
    - **What:** Five test functions, each printing `OK` or `WRONG`: `find_tool()` with a known and an unknown name, and `run_tool()` with a good call (returns `"32.0"`), an unknown tool and bad arguments (both return an error, no result) — all hand-made, no API key.
    - **Why:** It proves failures turn into readable error messages and only good results reach the Verifier — without any model call.
- `test_tools.py`
    - **What:** Two test functions that call the converter with `.invoke({...})`, each printing `OK` or `WRONG`: 18°C must give 64.4, and a bad argument (`celsius='hot'`) must be rejected by the tool's Pydantic check.
    - **Why:** A broken tool shows up here, with no network and no model — not as a strange agent answer later.
- `main.py`
    - **What:** Two functions. `print_steps(steps)` prints each tool call and its result; `main()` loads the config, builds the model, then loops: read a task, run `run_worker()`, print the step log and the answer, and turn a bad key, a rate limit or `MaxIterationsExceeded` into one clear message.
    - **Why:** One starting point that reads input and prints results. The agents and the flow live in other files.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 1 (18°C is 64.4°F)** — `tests/unit/test_tools.py`
    - **Check:** `celsius_to_fahrenheit.invoke({"celsius": 18})` returns `64.4`.
    - **Why:** the Worker trusts this number — the tool itself must be right before the model ever calls it.

    - **When:** after any change to `tools/converter.py`.
    - **If not:** a wrong formula would produce confident, wrong answers that look like real research.
- **case 2 (celsius='hot' refused)** — `tests/unit/test_tools.py`
    - **Check:** `{"celsius": "hot"}` raises `ValidationError`.
    - **Why:** `@tool` checks the argument's type before the function runs (Doc06) — bad arguments never reach your code.
    - **When:** same as case 1.
    - **If not:** a model's badly-typed call would crash inside the tool instead of becoming a clean error.
- **case 1 (known tool found)** — `tests/unit/test_agents.py`
    - **Check:** `find_tool(ALL_TOOLS, "celsius_to_fahrenheit")` is not `None`.
    - **Why:** every tool call the model makes starts with this lookup.
    - **When:** after any change to `agents/worker.py` or `ALL_TOOLS`.
    - **If not:** a tool missing from `ALL_TOOLS` would only show up as an "unknown tool" step in a real run.
- **case 2 (unknown tool gives None)** — `tests/unit/test_agents.py`
    - **Check:** `find_tool(ALL_TOOLS, "send_email")` is `None`.
    - **Why:** a made-up tool name must be reported, not matched to something wrong.
    - **When:** same as case 1.
    - **If not:** the Worker could run a different tool than the model asked for.
- **case 3 (good call gives 32.0)** — `tests/unit/test_agents.py`
    - **Check:** `run_tool()` on a good converter call returns `("32.0", None)`.
    - **Why:** the happy path: a result and no error.
    - **When:** same as case 1.
    - **If not:** a bug that drops the result would make every tool step look like a failure.
- **case 4 (unknown tool gives error)** — `tests/unit/test_agents.py`
    - **Check:** `run_tool()` on `send_email` returns no result and an error message.
    - **Why:** an unknown tool must become a readable observation, never a crash.
    - **When:** same as case 1.
    - **If not:** one hallucinated tool name would end the whole run with a traceback.
- **case 5 (bad arguments give error)** — `tests/unit/test_agents.py`
    - **Check:** `run_tool()` with `{"celsius": "x"}` returns no result and an error message.
    - **Why:** the `ValidationError` from case 2 must be caught inside `run_tool()`, so the model can try again.
    - **When:** same as case 1.
    - **If not:** a badly-typed call from the model would crash the loop instead of being retried.
- **case 2 (conversion)** — `tests/integration/test_live_agents.py`
    - **Check:** a Celsius-to-Fahrenheit question puts `celsius_to_fahrenheit` in the step log.

    - **Why:** the agent must use the tool for a calculation, not guess from memory.
    - **When:** at the end of the step, and after changing `prompts/worker.txt` or a tool's docstring.
    - **If not:** the agent could answer conversions from memory, with no step log to check.
- **case 3 (no tool needed)** — `tests/integration/test_live_agents.py`
    - **Check:** "What is the capital of France?" finishes with **zero** tool steps.
    - **Why:** calling tools for general knowledge costs time and money for nothing.
    - **When:** same as case 2.
    - **If not:** the agent could start calling tools for every question.
- **case 4 (step limit)** — `tests/integration/test_live_agents.py`
    - **Check:** a tool task with `max_iterations=1` raises `MaxIterationsExceeded`.
    - **Why:** the loop must **always** stop — this proves the limit is really enforced.
    - **When:** same as case 2, and after any change to the loop in `run_worker()`.
    - **If not:** a model that keeps asking for tools could loop (and bill you) forever.

Step 3 — Scaling to 3 Tools With a Hard Step Limit and Failure Recovery

Project: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — Step 3 of 4

- **Where you are now:** after Step 2 you have the finished Worker loop — with only one tool, and no tool that can fail.
- **What's new in this step:** a real-API tool (`tools/weather.py` + Doc02's `utils/http_client.py`), a tool that fails on purpose (`tools/docs_search.py`), a rewritten Worker prompt for 3 tools, and tests for them.
- **What stays the same:** the whole Worker loop from Step 2 — `run_worker()` , `run_tool()` , the step log and the limit. In `worker.py` , only the tool list grows. `main.py` doesn't change.
- **Why:** real tools call networks that fail, and real questions make the model choose between tools. This step proves the Step 2 loop survives both — without rewriting it.
- **How:** add the two tools to `ALL_TOOLS` . `run_tool()` already turns the failing search into an `Error: ...` observation, so the model retries it; the prompt tells it which tool fits which question.
- **Working after this step:** the full Worker — live weather, conversions and policy answers, recovery from the failing search, a printed step log, and all tests pass.

Read first: Doc07 — step limit, Doc07 Build Task, Doc06 — real API exercise, Doc06 — failure path.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add Doc02's `TransientHTTPError` / `PermanentHTTPError` to `exceptions.py`, and Doc02's `request_with_retry()` as `utils/http_client.py`.
2. Write `tools/weather.py` (Doc06's Open-Meteo tool, using `request_with_retry("GET", ...)`) and `tools/docs_search.py` (a small policy lookup that fails on its first call, using Doc07's `FlakyToolState`). Say in each description when *not* to use it.
3. Rewrite `prompts/worker.txt` for 3 tools: which questions need which tool, retry a failed tool once, and answer only from what the tools returned.
4. In `agents/worker.py`, import the two new tools and add them to `ALL_TOOLS` — nothing else changes.
5. Run the app: the new tools show up in the step log. `main.py` needs no change.
6. Add the tests: 2 more tool cases in `test_tools.py`, and 2 integration cases — a weather question uses `get_weather`, and the Worker recovers from the failing search.

Your files after Step 3:

```
ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/
├── src/
│   ├── agents/
│   │   └── worker.py        (changed) 2 tools added to ALL_TOOLS
│   ├── prompts/
│   │   └── worker.txt       (changed) rules for 3 tools
│   ├── tools/
│   │   ├── converter.py
│   │   ├── docs_search.py   (new) search_docs — fails on 1st call
│   │   └── weather.py       (new) get_weather — real API
│   ├── utils/
│   │   └── http_client.py   (new) request_with_retry()
│   └── exceptions.py        (changed) + 2 HTTP errors
└── tests/
    ├── integration/
    │   └── test_live_agents.py (changed) + cases 5-6
    └── unit/
        └── test_tools.py    (changed) + weather params, flaky search
```

Why these files (what and why):

- `agents/worker.py`
  - **What:** Written once in Step 2 and only extended after: `ALL_TOOLS` (one tool in Step 2, three from Step 3), an `AgentResult` dataclass (`final_answer`, `steps`), `find_tool(tools, name)`, `run_tool(call, tools)` (returns `(result, None)` or `(None, error)`), and `run_worker(model, task, max_iterations)`: bind the tools, then loop — invoke, append the reply, stop if there are no tool calls, otherwise run each call, append a `tool` message, record a step — and raise `MaxIterationsExceeded` after the last round.

    - **Why:** This is the agent that uses the tools. The step limit means it never runs forever, and the step log shows exactly how it reached its answer.
- `prompts/worker.txt`
    - **What:** This text file holds the worker prompt. The code loads it with `load_prompt("worker")`.
    - **Why:** Each agent's instructions are in one readable place, apart from the code.
- `tools/docs_search.py`
    - **What:** A `COMPANY_DOCS` dict, a `FlakyToolState` class with a `call_count`, and a `@tool` function `search_docs(query)` that raises `ConnectionError` on the first call and afterwards returns the policy text whose key appears in the query.
    - **Why:** It fails on purpose, so every run proves the Worker recovers from a tool failure instead of crashing.
- `tools/weather.py`
    - **What:** A `CITY_COORDINATES` dict, `build_weather_params(city)` (returns the latitude/longitude params or `None`), and a `@tool` function `get_weather(city)` that calls `request_with_retry("GET", WEATHER_URL, params=...)` and returns the temperature as text.
    - **Why:** The model can't know today's weather from its training. This gives it live data, with timeouts and retries from Doc02.
- `utils/http_client.py`
    - **What:** Doc02's `request_with_retry(method, url, max_attempts=5, **kwargs)`: a `for` loop that calls `requests.request(...)` with a timeout, returns `response.json()` on success, sleeps with `compute_backoff_delay()` and retries on timeouts, 429 and 5xx, and raises `PermanentHTTPError` on other 4xx.
    - **Why:** The weather tool gets timeouts and retries without writing them again.
- `exceptions.py`
    - **What:** Four error classes: `MissingConfigError` (Doc01), `TransientHTTPError` and `PermanentHTTPError` (Doc02), and `MaxIterationsExceeded` (Doc07), which carries the partial step log.
    - **Why:** Each problem can be caught by name. `MaxIterationsExceeded` carries the step log, so hitting the limit still shows where the Worker got stuck.
- `test_live_agents.py`
    - **What:** Step 2's cases plus a weather task that must use `get_weather`, and recovery from the failing search: at least one failed step and one successful one (cases 5-6).
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `test_tools.py`
    - **What:** Step 2's converter cases plus three new functions: `build_weather_params()` for Lahore (latitude 31.55) and for an unknown city (`None`), and the failing search — its first call raises `ConnectionError`, the second returns the refund policy (cases 3-5).

    - **Why:** A broken tool shows up here, with no network and no model — not as a strange agent answer later.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 3 (Lahore coordinates)** — `tests/unit/test_tools.py`
    - **Check:** `build_weather_params("Lahore")` returns latitude `31.55`.
    - **Why:** the weather tool is only as right as the coordinates it sends — checked here with no network.
    - **When:** after any change to `tools/weather.py`.
    - **If not:** a typo in a city's coordinates would return another city's weather, and look completely normal.
- **case 4 (unknown city gives None)** — `tests/unit/test_tools.py`
    - **Check:** `build_weather_params("Atlantis")` returns `None`.
    - **Why:** an unknown city must become a clear "no coordinates" message, not a bad API request.
    - **When:** same as case 3.
    - **If not:** an unknown city could send a broken request to the weather API.
- **case 5 (fails once, then works)** — `tests/unit/test_tools.py`
    - **Check:** after `search_state.call_count = 0`, the first `search_docs` call raises `ConnectionError` and the second returns the refund policy.
    - **Why:** the recovery test (case 6 below) only means something if the tool really fails once, every time.
    - **When:** after any change to `tools/docs_search.py`.
    - **If not:** case 6 could pass without any failure ever happening.
- **case 5 (weather task)** — `tests/integration/test_live_agents.py`
    - **Check:** a live weather question puts `get_weather` in the step log.
    - **Why:** with 3 tools, the model must pick the right one for live data.
    - **When:** at the end of the step, and after changing any tool's docstring.
    - **If not:** the agent could answer "the weather now" from memory.
- **case 6 (recovery)** — `tests/integration/test_live_agents.py`
    - **Check:** the refund question's step log has at least one failed and one successful step.
    - **Why:** a tool failing mid-run must become an observation the model works around — this step's main feature.
    - **When:** same as case 5.
    - **If not:** one flaky tool would end the whole run, or the agent would answer as if the failure never happened.

Step 4 — A Verifier Agent That Catches the Worker's Bad Answers (final: 2 agents)

Project: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — Step 4 of 4

- **Where you are now:** after Step 3 the Worker is complete — but its answer goes straight to the user, even if it misread a tool result.
- **What's new in this step:** `VerifierVerdict` in `schemas.py`, the Verifier (`agents/verifier.py` + `prompts/verifier.txt`), the Worker → Verifier flow with one retry (`services/research_service.py`), the compare script and its sample tasks.
- **What stays the same:** the whole Worker — its loop, tools and prompt don't change.
- **Why:** a second agent with its own prompt catches mistakes the first one can't see in its own work.
- **How:** the Verifier is a Doc05 LCEL chain with `with_structured_output(VerifierVerdict)`. It gets the question, the answer and only the tools' successful results. On a rejection, the Worker retries once with the reason; the answer is shown as verified or NOT verified.
- **Working after this step:** the full project — every answer is labelled, a wrong one is caught, and all tests pass.

Read first: Doc05 — structured parser exercise, and as a look ahead, Doc11 — Patterns (the generator → critic row).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `VerifierVerdict` (`approved: bool`, `reason: str`) in `schemas.py`.
2. Write `prompts/verifier.txt`: a strict checker that did *not* write the answer, with `{question}`, `{evidence}` and `{answer}` placeholders.
3. Write `agents/verifier.py`: `build_verifier_chain(model)`, `collect_evidence(steps)` (only successful tool results — not the Worker's retries) and `verify(chain, question, result)`, which logs the verdict.
4. Write `services/research_service.py`: `answer_task()` runs the Worker, then the Verifier; on a rejection, it reruns the Worker **once** with the reason added, and verifies again against the original question.
5. Update `main.py`: build the Verifier chain once, call `answer_task()`, and label each answer verified or NOT verified.
6. Add the tests: `collect_evidence()` in the unit test, and in the integration test a hand-made wrong answer that **must be rejected** and a right one that must pass.
7. Write `data/sample/tasks.json` and `scripts/compare_with_tools.py`, and run it: memory answer vs. verified answer, side by side.

Your files after Step 4 (final): the full tree in Project Structure. What changed:

```
ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/
├── data/sample/
```

```
│   └── tasks.json           (new) sample tasks for the script
├── scripts/
│   └── compare_with_tools.py (new) memory vs. verified answer
├── src/
│   ├── agents/
│   │   └── verifier.py      (new) build_verifier_chain(), verify()
│   ├── prompts/
│   │   └── verifier.txt     (new)
│   ├── services/
│   │   ├── __init__.py      (new)
│   │   └── research_service.py (new) answer_task()
│   └── schemas.py           (new) VerifierVerdict
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py (changed) + cases 7-8
│   └── unit/
│       └── test_agents.py   (changed) + collect_evidence()
└── main.py              (changed) Worker + Verifier, labelled answer
```

Why these files (what and why):

- `data/sample/tasks.json`
  - **What:** A JSON object with a `"tasks"` list of sample questions.
  - **Why:** The script uses the same inputs every run, so you can compare results over time.
- `scripts/compare_with_tools.py`
  - **What:** One `main()` that reads `tasks.json` and, for each task, prints the direct chain's answer, then `answer_task()`'s answer and verdict.
  - **Why:** It shows the project's whole point in one run — a guess next to a checked answer — for you to judge.
- `agents/verifier.py`
  - **What:** Three functions: `build_verifier_chain(model)` returns `prompt | model.with_structured_output(VerifierVerdict)`; `collect_evidence(steps)` joins only the successful tool results; `verify(chain, question, result)` invokes the chain and logs the verdict.
  - **Why:** A second agent with its own prompt checks the answer against the tools' evidence, and catches a confident wrong answer before the user sees it.
- `prompts/verifier.txt`
  - **What:** This text file holds the verifier prompt. The code loads it with `load_prompt("verifier")`.
  - **Why:** Each agent's instructions are in one readable place, apart from the code.
- `services/research_service.py`
  - **What:** One function, `answer_task(model, verifier_chain, task, max_iterations)`. It runs `run_worker()`, then `verify()`. If the answer is rejected, it adds the reason to the

task, runs the Worker **once** more, verifies again against the original task, and returns `(result, verdict)`.
    - **Why:** The whole handoff between the two agents is one testable function, and a rejected answer gets exactly one chance to be fixed.
- `schemas.py`
    - **What:** One Pydantic class, `VerifierVerdict`, with `approved: bool` and `reason: str`.
    - **Why:** The Verifier's answer is a real True/False the code can use. Without it you'd search the text for "approved" — and "NOT approved" would pass.
- `test_live_agents.py`
    - **What:** Test functions that call the real model (and the weather API), added step by step: the no-tools chain, the right tool for a clear task, no tool for general knowledge, the step limit, a weather task, recovery from the failing search, and the Verifier rejecting a wrong answer and approving a right one. Each prints `OK` or `WRONG`.
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `test_agents.py`
    - **What:** One test function per case, each printing `OK` or `WRONG`: hand-made tool-call dicts for `find_tool()` and `run_tool()` (known tool, unknown tool, good call, unknown-tool call, bad arguments), and hand-made steps for `collect_evidence()` (a failed step left out, no steps).
    - **Why:** It proves failures turn into readable error messages and only good results reach the Verifier — without any model call.
- `main.py`
    - **What:** `main()` loads the config, builds the model and the Verifier chain, then loops: read a task, call `answer_task()`, print the step log with `print_steps()`, and print the answer marked verified or NOT verified. It catches `MaxIterationsExceeded` (prints the partial log) and the OpenAI key and rate-limit errors.
    - **Why:** One starting point that reads input and prints results. The agents and the flow live in other files.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 6 (failed step left out)** — `tests/unit/test_agents.py`
    - **Check:** `collect_evidence()` on one failed and one good step includes the good result and not the failed one.
    - **Why:** the Verifier must only see what the tools really returned.
    - **When:** after any change to `agents/verifier.py`.
    - **If not:** an error message could be treated as evidence, and a wrong answer approved.
- **case 7 (no steps)** — `tests/unit/test_agents.py`
    - **Check:** `collect_evidence([])` returns `"(no tools were used)"`.

    - **Why:** a from-memory answer must be judged with an honest "no evidence", not an empty string.
    - **When:** same as case 6.
    - **If not:** the Verifier prompt would get a blank evidence section and guess what it means.
- **case 7 (wrong answer)** — `tests/integration/test_live_agents.py`
    - **Check:** evidence says 64.4°F, the answer says 90°F — `verify()` returns `approved=False`.
    - **Why:** a Verifier that has never rejected anything hasn't been tested. This is the most important case in the project.
    - **When:** at the end of the step, and after changing `prompts/verifier.txt` or `VerifierVerdict`.
    - **If not:** a Verifier that approves everything would look exactly like one that works.
- **case 8 (right answer)** — `tests/integration/test_live_agents.py`
    - **Check:** the same evidence with the right answer (64.4°F) gives `approved=True`.
    - **Why:** the opposite check — a Verifier that rejects everything is just as broken.
    - **When:** same as case 7.
    - **If not:** every good answer would be retried for nothing, doubling the cost. Not a test: `scripts/compare_with_tools.py` prints from-memory and verified answers side by side for **you** to read. Tests check facts; only a person can judge whether the verified answer is really better.

Final Deliverable: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — a 2-agent system in a proper package layout. A Worker solves tool-using tasks with a hard limit and failure recovery. An independent Verifier checks every answer against the tools' evidence before the user sees it.

Why this really is multi-agent (said plainly): a simple **generator → critic** pair (Doc11) — no shared graph state yet (Project 3), no supervisor over several roles yet (Project 4). But it is two independent agents with a real handoff, and a real chance for the Verifier to catch something.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 4 is done, this is how the finished project is run, tested and configured.

Configuration

Settings are read from `.env` (never committed):

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key

Variable	Required	Default	Meaning
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

The model (`gpt-4o-mini`) is set in one place: `MODEL_NAME` in `src/models/llm.py`. The step limit is `MAX_ITERATIONS` in `main.py`. Prompts are in `src/prompts/`. Weather works for the cities listed in `tools/weather.py`.

Usage

Run from the repo root:

```
python main.py
```

Ask about the weather (in °C or °F), the company's refund, shipping or warranty policy, or anything general. Type `/quit` to exit.

To compare a from-memory answer with the verified one for each sample task:

```
python -m scripts.compare_with_tools
```

Running the Tests

Run from the repo root:

```
# unit tests — no API key, no network
python -m tests.unit.test_config
python -m tests.unit.test_tools
python -m tests.unit.test_agents

# integration tests — real API calls, needs OPENAI_API_KEY
python -m tests.integration.test_live_agents
```

Every test case prints `OK` or `WRONG`. The unit tests check the config, the tools and the Worker's helpers. The integration tests check: the no-tools chain, the right tool for a clear task, no tool for general knowledge, recovery from a failing tool, the step limit, and that the Verifier **rejects** a deliberately wrong answer and approves a right one.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] `main.py` at the repo root, the code in `src/`; runs with `python main.py`

- [ ] Every folder has one clear job, and you can say why it exists
- [ ] 3 tools on the Worker, each with a description that says when *not* to use it
- [ ] Tool arguments are checked by Pydantic (`@tool`) before the tool runs
- [ ] A hard step limit — `MaxIterationsExceeded` with the partial log, never an endless loop
- [ ] The failing search is recovered from in a real run, not crashed on or made up
- [ ] The Verifier runs on every answer, with its own separate prompt
- [ ] At least one test where the Verifier rejects a wrong answer
- [ ] A step log for every run, and the Verifier's verdict logged separately
- [ ] Every test case prints `OK` — `tests/unit/` with no API key, `tests/integration/` with one
- [ ] You wrote each step's tests yourself, and can say for each one what goes wrong without it
- [ ] `.env` is not in git
- [ ] You can explain why the Worker chose each tool, and why the Verifier approved or rejected

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and put your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status                     # .env and .venv/ must NOT appear
git add .
git commit -m "ResearchHand: tool-using Worker + Verifier agents"
```

3. On GitHub, create a new repo named exactly `ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of README.md. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started. Track your progress in ../PROGRESS.md.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**